

There should be one—and preferably only one—obvious way to do it. This is a broadside against the Perl programming language’s motto, “There’s more than one way to do it!” It turns out that having three or four different ways to write code that does the same task is a double-edged sword: you have flexibility in how you write code, but now you have to learn every possible way it could have been written to read other people’s code. This flexibility isn’t worth the increased effort needed to learn a programming language.

Although that way may not be obvious at first unless you’re Dutch. This line is a joke. Guido van Rossum, the creator of Python, is Dutch.

Now is better than never. Although never is often better than *right* now. These two aphorisms tell us that code that runs slowly is obviously worse than code that runs quickly. But it’s better to have to wait for your program to finish than to finish it too early with incorrect results.

If the implementation is hard to explain, it’s a bad idea. If the implementation is easy to explain, it may be a good idea. Many things get more complicated over time: tax laws, romantic relationships, Python programming books. Software is no different. These two aphorisms remind us that if code is so complicated as to be impossible for programmers to understand and debug, it’s bad code. But just because it’s easy to explain a program’s code to someone else doesn’t mean it isn’t bad code. Unfortunately, figuring out how to make code as simple as possible, and not any simpler, is hard.

Namespaces are one honking great idea—let’s do more of those! Namespaces are separate containers for identifiers to prevent naming conflicts. For example, the `open()` built-in function and the `webbrowser.open()` function have the same name but refer to different functions. Importing `webbrowser` doesn’t overwrite the built-in `open()` function because both `open()` functions exist in different namespaces: the built-in namespace and the `webbrowser` module’s namespace, respectively. But keep in mind that flat is better than nested: as great as namespaces are, you should make them only to prevent naming conflicts, not to add needless categorization.